Grade-Forge — Implementation Specifications

Summary (what we use)

- **Frontend**: React (TypeScript) SPA
- **Backend**: Spring Boot (Java 21) REST API + JWT auth
- **Database**: PostgreSQL
- **File storage**: AWS S3
- **Async jobs**: RabbitMQ
- **Code execution**: Docker-based test runs
- **Grading/report pipeline**: Python
- **Docs**: VitePress served under `/docs`
- **Deployment**: Docker container on AWS EC2 (CI builds and deploys over SSH)

Team members

- Bipin Shrestha
- Damir Filaretov
- Balkrishna Basnet
- Bishwo Dahal

Languages / frameworks (and how they're used)

- **TypeScript + React 18 + Vite 6**
  - **Purpose**: Single Page Application (SPA) UI for students/faculty/admins.
  - **HTTP**: Axios client with `Authorization: Bearer <token>`.
  - **Editor**: Monaco editor for code workspace.
  - **UI**: MUI + Radix UI + Tailwind.
  - **Routing**: React Router.
  - **Client storage**:
    - **JWT + user session**: `sessionStorage`
    - **Workspace state (files/tabs/contents)**: IndexedDB
- **Java 21 + Spring Boot 4.0.2**
  - **Purpose**: REST API, authentication/authorization, database access, background job consumers, static hosting for SPA + docs.
  - **Web**: Spring MVC

- **Security**: Spring Security + JWT (JJWT library)
  - Token parsed from `Authorization: Bearer ...`
  - Role/authority routing via Spring Security configuration.
- **Database**: Spring Data JPA + PostgreSQL driver
- **Async jobs**: RabbitMQ via Spring AMQP
- **API docs**: OpenAPI/Swagger UI via Springdoc
- **Email**: SMTP via Spring Mail (optional; configured via env vars)
- **AWS S3**: AWS SDK v2 for file storage/integration

- **Python 3**
  - **Purpose**: "grader report" pipeline (similarity/plagiarism/AI signals) executed by the backend.
  - **Key deps**:
    - `copydetect` (code similarity)
    - `numpy`, `scikit-learn`, `joblib` (authorship/ML inference pieces)
  - **Entrypoint**: reads an `input.json`, prints output JSON.

- **VitePress**
  - **Purpose**: Product documentation site.
  - **Prod serving**: built into backend static resources and available under `/docs/`.

- **Docker / Docker Compose**
  - **Purpose**: local RabbitMQ, and production builds/deploys of a single container image.
  - **Run-tests**: backend can invoke the `docker` CLI to run student code in containers; production runs with Docker socket access.

System architecture (modules)

- **Frontend**
  - SPA UI (role-based screens for students/faculty/admins)
  - Assignment coding workspace (Monaco-based)

- **Backend**
  - REST endpoints under `/api/v1/**` (examples: auth, assignments, submissions, grading)
  - Security: JWT + role authorities
  - Background processing:
    - test-run job consumer(s)
    - grader-report job consumer(s)
  - Static hosting:

    - SPA assets (React build)
    - Docs under `/docs/`

- **Grader pipeline**
    - Called by backend for report generation; optional LLM evidence via Ollama-compatible endpoint (configurable).

Data collection (high level)

- **User input**:
    - signup/login, password updates/resets
    - profile updates and admin/faculty management APIs (role-restricted endpoints under `/api/v1/**`)
- **Coursework**:
    - assignment creation/updates, rubric/test suite management
    - student submissions (multipart file uploads)
    - grading workflows (faculty / grading assistant)
- **Execution & reports**:
    - run-tests requests upload files (optionally stdin)
    - grader-report generation consumes submissions and outputs reports

---

Data storage (where things live)

- **PostgreSQL** (primary relational store)
    - connection configured by:
        - `SPRING_DATASOURCE_URL`
        - `SPRING_DATASOURCE_USERNAME`
        - `SPRING_DATASOURCE_PASSWORD`
- **AWS S3** (object storage)
    - bucket + credentials configured by:
        - `CLOUD_AWS_S3_BUCKET_NAME`
        - `CLOUD_AWS_CREDENTIAL_ACCESS_KEY`
        - `CLOUD_AWS_CREDENTIAL_SECRET_KEY`
        - `CLOUD_AWS_REGION`
- **RabbitMQ** (async queues)

    - local dev via Docker Compose
    - queue names:
        - `execution.queue.test-run-jobs=test-run-jobs`
        - `execution.queue.grader-report-jobs=grader-report-jobs`

- **Browser storage**
    - `sessionStorage` : JWT + user snapshot
    - IndexedDB: code workspace persistence per assignment

Authentication / authorization

- **Authentication**: JWT bearer tokens (`Authorization: Bearer <token>`)
    - client attaches token via HTTP interceptor
    - server validates/parses token via JWT filter
- **Authorization**: Spring Security authorities (roles)
    - roles observed in client code include:
        - `STUDENT` , `FACULTY` , `GRADING_ASSISTANT` , `UNIVERSITY_ADMIN` , `SYSTEM_ADMIN`

External integrations / optional services

- **Canvas** (optional)
    - configured by env vars:
        - `CANVAS_BASE_URL` , `CANVAS_TOKEN`
- **Email** (optional)
    - configured by env vars: `SPRING_MAIL_*`
- **Ollama (LLM)** (optional, for grader "AI evidence signals")
    - configured by `GRADER_LLM_AI_SIGNAL_*`
    - in production, the container is configured to reach an Ollama service running on the host (or another private host).

Deployment / hosting (location + topology)

Local development

- **RabbitMQ**: `docker compose up -d`
- **Backend**: `mvn spring-boot:run` → `http://localhost:8080`
- **Frontend**: `npm run dev` → typically `http://localhost:5173`
- **Docs**:
  - VitePress dev server OR
  - build docs into Spring static docs and serve under `/docs/`

Production (as implemented in this repo)

- **Compute (app)**: **AWS EC2**, running Docker containers
  - CI builds and pushes a Docker image, then deploys to EC2 over SSH
  - By default, branch-to-port mapping:
    - `main` → host port `8080`
    - `production` → host port `8081`
- **Database**: external PostgreSQL (typically **RDS Postgres**) via `SPRING_DATASOURCE_*` secrets.
- **Object storage**: **AWS S3** via `CLOUD_AWS_*` secrets.
- **Message broker**: RabbitMQ container (started/ensured by deploy workflow) or an external RabbitMQ host.
- **Optional reverse proxy / HTTPS**: Nginx on EC2

Build & packaging

- **Single Docker image** builds:
  - React SPA → copied into Spring static resources
  - VitePress docs → copied into the app's static docs directory
  - Spring Boot JAR (single runnable artifact)
  - Python grader + ML training deps installed into a venv inside the image

---

Operational notes / "what's required" for production

- **Required**:
  - Postgres credentials (`SPRING_DATASOURCE_*`)
  - S3 bucket + credentials (`CLOUD_AWS_*`)

    - RabbitMQ (either external or the workflow-managed container)
    - Docker socket access for “run tests” if you enable that feature in production

- **Optional**:
    - Ollama host/service for LLM evidence (`GRADER_LLM_AI_SIGNAL_*`)
    - Canvas integration (`CANVAS_*`)
    - Email (`SPRING_MAIL_*`)